Avinash. M
(192521196)

SIMATS ENGINEERING
ASSESSMENT TOOL 2

38
40

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately 2	Minor missing elements 1	Partial understanding 1	Incorrect interpretation 2
Algorithm Design	30%	Complete, correct, and logically sound solution 4	Minor logical gaps 2	Partially correct logic 2	Incorrect approach 1
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules 2	Minor inefficiencies 2	Limited or basic usage 2	Incorrect or missing constructs 2
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow 2	Mostly clear with minor issues 1	Difficult to follow in parts 2	Poorly structured and unclear 2
Completeness	20%	Handles all cases; optimized approach 2	Handles most cases; reasonable efficiency 2	Handles basic cases only 2	Incomplete or inefficient 2

9 8 4 9

COMPUTER NETWORKS
PSEUDOCODE WRITING TASK
REPORT

Course Code: CSA07
Course Outcome Covered: CO5
Assessment Tool 2
Simulation Tools & Network Performance

Complete Question & Answer Document

Q1: In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

Answer:

```
Algorithm Network_Performance_Evaluator
Inputs:
   total_data_bits = 8000
   total_time_sec = 4
   packets_sent = 200
   packets_received = 180

BEGIN
   // Step 1: Input Validation
   IF total_time_sec == 0 OR packets_sent == 0 THEN
      PRINT "Error: Time or packets sent cannot be zero."
      EXIT ALGORITHM
   END IF

   // Step 2: Calculate Network Metrics
   throughput_bps = total_data_bits / total_time_sec

   packets_lost = packets_sent - packets_received
   packet_loss_percentage = (packets_lost / packets_sent) * 100

   // Step 3: Display Initial Results
   PRINT "Calculated Network Throughput: " + throughput_bps + " bps"
   PRINT "Calculated Packet Loss: " + packet_loss_percentage + "%"
```

```
// Step 4: Network Classification Logic
IF throughput_bps > 1500 AND packet_loss_percentage < 10 THEN
   network_status = "Efficient"
ELSE
   network_status = "Inefficient"
END IF

// Step 5: Output Complete Summary
PRINT "----------------------------------------"
PRINT "   Complete Network Performance Summary   "
PRINT "----------------------------------------"
PRINT "Total Data Received: " + total_data_bits + " bits"
PRINT "Total Time: " + total_time_sec + " seconds"
PRINT "Throughput: " + throughput_bps + " bps"
PRINT "Packet Loss Percentage: " + packet_loss_percentage + "%"
PRINT "Network Classification: " + network_status
END
```

Q2: An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

Answer:

```
Algorithm Bandwidth_Management_System
Inputs:
   network_links_list = [Link1, Link2, Link3, ...]
   monitoring_interval = 5 seconds

BEGIN
   // Start continuous loop to monitor all links
   WHILE System_is_Running DO
      FOR EACH link IN network_links_list DO
         // Collect current bandwidth usage
         current_utilization = get_link_utilization(link)

         // Store the values in a list for tracking
         link_usage_history[link.id].append(current_utilization)

         // Check if bandwidth utilization exceeds threshold
         IF current_utilization > 80 THEN
            PRINT "ALERT: Link " + link.id + " Overloaded (" + current_utilization +
"%)!"

            // Identify an available backup link
            backup_link = find_available_backup_link()

            // Recommend and execute traffic switch
            PRINT "Action Recommended: Switching traffic to " + backup_link.id
            switch_traffic(link, backup_link)
         ELSE
```

```
            PRINT "Status: Link " + link.id + " operating normally."
        END IF
    END FOR

    // Wait before initiating the next collection cycle
    WAIT(monitoring_interval)
  END WHILE
END
```

Explanation:

This algorithm significantly supports efficient bandwidth management by automating the constant real-time surveillance of all available network links. By continuously looping through the links and storing utilization data, administrators can track historical trends and traffic patterns. The condition that detects utilization exceeding 80% proactively identifies potential congestion before it leads to severe packet delays or drops. By instantly recommending and initiating a traffic switch to an alternate backup link, the algorithm guarantees seamless load balancing. This preventative strategy ensures that no single link becomes a bottleneck, maintaining smooth and optimized network communication.

Q3: A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

Answer:

```
Algorithm Adaptive_Optimal_Path_Selector
Inputs: hq_to_branches_paths = [Path1, Path2, Path3, ...]

BEGIN
  WHILE System_Active DO
    best_communication_path = NULL
    highest_quality_score = -9999

    FOR EACH path IN hq_to_branches_paths DO
      // Periodically collect dynamic link metrics
      bandwidth = monitor_bandwidth(path)
      delay = monitor_delay(path)
      utilization = monitor_utilization(path)

      // Store collected values in historical lists
      path.bandwidth_history.add(bandwidth)
      path.delay_history.add(delay)
      path.utilization_history.add(utilization)

      // Detect failed or overloaded links
```

```
IF path_status(path) == "FAILED" OR utilization >= 100 THEN
   PRINT "Critical Alert: Path " + path.id + " failed/overloaded!"
   generate_administrator_alert(path)
   CONTINUE // Skip to evaluate next path
END IF

// Calculate comprehensive Link Quality Score
link_quality_score = (bandwidth * 0.5) - (delay * 0.3) - (utilization * 0.2)

// Identify the optimal path with highest score
   IF link_quality_score > highest_quality_score THEN
      highest_quality_score = link_quality_score
      best_communication_path = path
   END IF
END FOR

// Automatically re-route if optimal path changes
IF best_communication_path != current_active_path THEN
   PRINT "Network changed. Selecting new optimal path..."
   current_active_path = best_communication_path
   route_traffic(current_active_path)
   PRINT "Active Path: " + current_active_path.id
END IF

WAIT(10 seconds) // Interval before next collection
  END WHILE
END
```

Explanation:

This comprehensive algorithm dramatically improves routing efficiency by calculating a dynamic Link Quality Score instead of relying on rigid shortest-path protocols. By integrating bandwidth, delay, and real-time utilization, it ensures traffic flows through the most capable routes. Network reliability and fault tolerance are strictly maintained by detecting failures instantly, issuing alerts, and automatically failing over to the next optimal path without human intervention. This proactive and continuous adaptation guarantees that communication remains stable and robust, successfully responding to fluctuating network conditions throughout the day.